Reviewer Pack — Minimal Order of Modular Forms and Circuit Monotone Width Co…

Entry bd06a87fc8f9 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 21:12:37 UTC

Minimal Order of Modular Forms and Circuit Monotone Width Correlation

Entry ID: `bd06a87fc8f9` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-01 21:12:37 UTC

1. Conjecture

Field A (mathematical branch): Modular Form Theory **Field B** (complexity object): Boolean Circuits

Statement:

> For every modular form f with level N and weight k, the minimal order (ord_min(f)) of its q-expansion is linearly correlated with the monotone width (w_m(f)) of the corresponding circuit, such that ord_min(f) = Θ(w_m(f)).

Rationale (proposer's reasoning):

> Modular forms are known to exhibit rich arithmetic structures, and their properties have been extensively studied in number theory. The correlation between these structures and the complexity of circuits could reveal new insights into both fields. If true, this conjecture would establish a bridge between algebraic objects with complex arithmetic properties and computational complexity measures.

Taxonomy category: `MODULAR_FORMS` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `c70a0f390c8d98be`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if at least 80% of the correlations between ord_min(f) and w_m(f) across all seeds are within a factor of 1 ± 3 standard deviations of linear correlation with a p-value ≤ 0.05.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `Minimal order modular forms AND circuit monotone width` - `Modular form q-expansion level N weight k CORRELATES WITH circuit monotone width` - `ord_min(q-expansion) Θ(w_m(circuit))`

Top relevant hits considered: - [http://arxiv.org/abs/2412.20260v2] Functorial, operadic and modular operadic combinatorics of circuit algebras - [http://arxiv.org/abs/alg-geom/9506017v2] Minimal Siegel modular threefolds - [http://arxiv.org/abs/1806.05207v2] Interpolated sequences and critical L-values of modular forms - [http://arxiv.org/abs/2604.05701v1] Measurement of the CKM angle γ in $B^{\pm} \to D(\to K^0_S h'^+ h'^-)h^{\pm}$ dec - [http://arxiv.org/abs/2510.17487v1] Directional Search for Persistent Gravitational Waves: Results from the First Part of LIGO-Virgo-KAGRA's Fourth Observin - [http://arxiv.org/abs/2601.07595v3] Deep Search for Joint Sources of Gravitational Waves and High-Energy Neutrinos with IceCube During the Third Observing R - [http://arxiv.org/abs/1110.2998v1] Equivalent Quantum Circuits - [http://arxiv.org/abs/1512.07240v3] The block-ZXZ synthesis of an arbitrary quantum circuit

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    # Set seed for reproducibility
    random.seed(seed)

    # Define a simple function to generate modular forms and circuits
    def generate_modular_form(level, weight):
        # Placeholder for actual modular form generation logic
        return level * weight

    def construct_circuit(modular_form_value):
        # Placeholder for actual circuit construction logic
        return modular_form_value + 1

    def compute_minimal_order(modular_form_value):
        # Placeholder for actual minimal order computation logic
        return abs(modular_form_value)

    def compute_monotone_width(circuit):
        # Placeholder for actual monotone width computation logic
        return len(circuit)
```

```
    # Define the range of levels and weights to test
    levels = [5, 10, 15, 20, 30, 40]
    weights = [1, 2, 3, 4, 5]

    # Initialize variables for the trial
    metric_values = []
    instances_tested = 0
    n_max = 0

    # Run multiple trials with different levels and weights
    for level in levels:
        for weight in weights:
            modular_form_value = generate_modular_form(level, weight)
            circuit = construct_circuit(modular_form_value)
            minimal_order = compute_minimal_order(modular_form_value)
            monotone_width = compute_monotone_width(circuit)

            # Record the metric value
            metric_values.append(minimal_order / monotone_width)
            instances_tested += 1
            n_max = max(n_max, level)

    # Compute the mean and standard deviation of the metric values
    mean_metric_value = sum(metric_values) / len(metric_values)
    std_metric_value = math.sqrt(sum((x - mean_metric_value) ** 2 for x in metric_values) / len(me

    # Check if the conjecture holds based on the correlation
    correlations = [(x, y) for x, y in zip(metric_values[:-1], metric_values[1:])]
    correlation_coefficient = sum((x - mean_metric_value) * (y - mean_metric_value) for x, y in co
    p_value = 2 * (1 - math.erf(abs(correlation_coefficient) * math.sqrt(len(correlations) / 2)))

    # Determine if the conjecture holds
    conjecture_holds = abs(correlation_coefficient) >= 0.7 and p_value <= 0.05

    # Return the trial results as a dictionary
    return {
        "metric_name": "Correlation between minimal order and monotone width",
        "metric_value": correlation_coefficient,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": "" if conjecture_holds else "mapping_undefined"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    # Compute the mean and standard deviation of the metric values
```

```
    mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
    std_metric_value = math.sqrt(sum((result["metric_value"] - mean_metric_value) ** 2 for result

    # Compute the fraction of seeds where the conjecture holds
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    # Determine the final result
    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_fai
    else:
        print(f"RESULT: INCONCLUSIVE reason=insufficient_evidence")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_898a01fe.py", line 89, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_898a01fe.py", line 54, in run_trial
    monotone_width = compute_monotone_width(circuit)
                     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_898a01fe.py", line 37, in compute_monot
    return len(circuit)
           ^^^^^^^^^^^^
TypeError: object of type 'int' has no len()
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed before producing data, which means it did not complete its execution to provide results for the correlation analysis. | next: Re-run the test with proper error handling and ensure that it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15518
2	preregistration	ollama_remote	glm4:latest	0	0	9636
3	novelty	ollama_remote	glm4:latest	0	0	11725
4	novelty	ollama_remote	glm4:latest	0	0	12528
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	30014
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	24713
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16035
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11604
9	judge	ollama_remote	glm4:latest	0	0	18636

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 150408 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/bd06a87fc8f9.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/bd06a87fc8f9.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/bd06a87fc8f9.tar.gz` (if generated)